



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea Magistrale in Ingegneria Informatica

Tesi di Laurea Magistrale in Ingegneria Informatica

*Integrazione del Protocollo TEC Tree per la
Sicurezza e Autenticazione della Memoria
Off-Chip con il Protocollo di Coerenza
Tardis-TSO*

Anno Accademico 2025/2026

Relatore

Ch.mo prof. Alessandro Cilardo

Candidato

Fuorto Simone

matr. M63001564

Abstract

Con l'aumentare della gamma di servizi forniti dai sistemi embedded, cresce anche la quantità di dati sensibili che essi manipolano. Di conseguenza, esistono attualmente forti incentivi per gli aggressori che desiderano trarre profitto da attacchi a questi sistemi. Gli attacchi sui bus consentono di recuperare i dati in transito da o verso la memoria, sollevando il problema della riservatezza dei dati. In generale, per garantire la riservatezza vengono implementati schemi di cifratura. Tuttavia, nella maggior parte dei casi, la sola crittografia non è sufficiente quando un avversario è in grado di manipolare i dati senza essere rilevato (vedi *bus probing*). L'integrità della memoria è quindi un obiettivo primario per la protezione di una piattaforma informatica contro attaccanti attivi. L'obiettivo di questo lavoro è l'implementazione di un protocollo in grado di garantire integrità e confidenzialità grazie alla cifratura della memoria *off-chip* basata su *Tamper-Evident Counter Tree* e di valutarne le performance in relazione ai costi di realizzazione introdotti.

Indice

Abstract	i
1 Introduzione	1
1.1 Simulatore Gem5	1
1.2 Protocollo di Sicurezza e Integrità	2
1.3 Workflow	2

Capitolo 1

Introduzione

1.1 Simulatore Gem5

Il simulatore gem5 è una piattaforma modulare basata su eventi discreti utilizzata nell'ambito dell'architettura dei calcolatori. La composizione modulare del sistema offre la possibilità di riconfigurare, estendere o sostituire ciascun componente dell'architettura adattandosi a molteplici esigenze di simulazione. Tale tool permette di emulare diversi sistemi di calcolo, permettendo la simulazione di diverse architetture come *Alpha*, *ARM*, *MIPS*, *Power*, *SPARC*, *RISC-V* e *x86* a 64 bit. Il simulatore è scritto principalmente in linguaggio C++ e Python e offre spiccata flessibilità nel costruire sistemi di memoria grazie a cache e crossbar ma soprattutto grazie al modulo Ruby, utile per una modellazione avanzata. Le modalità di simulazione previste da gem5 sono principalmente due: **full system mode**, simulando dispositi-

vi e sistemi operativi completi o **syscall emulation** per simulazioni circoscritte nello spazio utente.

1.2 Protocollo di Sicurezza e Integrità

Il nucleo centrale di questa ricerca si focalizza sull'impiego del modulo di simulazione Ruby, uno strumento progettato per simulare in modo estremamente accurato il comportamento delle interconnessioni e dei sottosistemi di memoria nei processori moderni. Grazie alla sua spiccata flessibilità nel modellare le gerarchie di memoria e i relativi protocolli, Ruby costituisce l'ambiente ideale per la progettazione e il testing di nuove soluzioni. In questo contesto, il modulo verrà impiegato per concretizzare il protocollo di sicurezza ed integrità proposto, l'implementazione del quale costituisce il contributo principale del presente lavoro.

1.3 Workflow



Figura 1.1: Workflow del processo.

Il workflow è articolato in quattro fasi e prevede:

- Analisi dei Fondamenti Teorici su TEC-TREE: in questa fase verranno approfondite le specifiche del protocollo di Autenticazione della memoria.
- Progettazione del Protocollo: sulla base dei fondamenti teorici si procederà con la progettazione del protocollo mediante la definizione di una macchina a stati finiti, messaggi ed eventi.
- Implementazione del Protocollo: gli elementi progettati nella fase precedente saranno tradotti nella sintassi del linguaggio del simulatore gem5 e opportunamente integrati con il protocollo di coerenza preesistente.
- Validazione e Analisi Prestazionale: si procederà alla modellazione e configurazione di un'architettura multicore all'interno dell'ambiente di simulazione, al fine di eseguire benchmark multithread. Tali scenari di test permetteranno di analizzare differenti casi d'uso e di condurre uno studio quantitativo sull'impatto prestazionale, sull'efficacia e sulla scalabilità del protocollo di sicurezza sviluppato.

Bibliografia